Bioinformatics, YEAR, pp. 1–8
DOI added during production
Published: Date added during production
Paper

MCS-ELA: Multi-Code-Style Supervision and Entity-Level Alignment for Generative Biomedical Named Entity Recognition

Peiwen Gong,[1] Jinzhong Ning,[1,*] Wenxuan Mu,[1] Zitong Li,[1] Ling Luo[2] and Yijia Zhang[1,*]

[1] Information Science and Technology College, Dalian Maritime University, 116026, Dalian, China
[2] School of Computer Science and Technology, Dalian University of Technology, 116024, Dalian, China

*Corresponding authors: jingjinzhong@dlmu.edu.cn; zhangyijia@dlmu.edu.cn

Abstract

Motivation: Biomedical named entity recognition (BioNER) aims to identify entity mentions and their types in biomedical text. However, the structural priors acquired by large language models during pretraining remain underexplored in generative BioNER. Single-reference supervision further limits learning from equivalent outputs, while sequence-level likelihood optimization provides only indirect supervision for entity boundary and type correctness.

Results: We introduce Multi-Code-Style Supervision and Entity-Level Alignment (MCS-ELA) for generative biomedical named entity recognition. Semantically equivalent references in Python, Java and C++ styles, together with entity-order augmentation, diversify supervision; verifiable entity and format rewards guide optimization, and multi-style voting aggregates predictions. Across six settings from five biomedical corpora, MCS-ELA with Qwen3-8B as its backbone outperforms both evaluated generative baselines in all settings and achieves the highest micro-F1 among compared methods in five. Ablation studies support the contributions of each training component.

Availability and Implementation:

1. Introduction

Biomedical Named Entity Recognition (BioNER) aims to identify entities such as diseases, drugs, and genes from unstructured biomedical text (Habibi et al., 2017). As a fundamental task in biomedical text mining, BioNER supports downstream applications such as information retrieval (Xu et al., 2024) and relation extraction (Lai et al., 2021). Compared with general-domain NER, BioNER is particularly challenging due to specialized terminology, abbreviations, and complex multi-word entities, which complicate entity boundary detection and type classification (Song et al., 2021).

Traditional BioNER methods typically formulate the task as sequence labeling, leveraging biomedical pretrained language models (PLMs), such as BioBERT (Lee et al., 2020), PubMedBERT (Gu et al., 2021), and BioLinkBERT (Yasunaga et al., 2022), to predict token-level entity labels. Recently, large language models (LLMs) have enabled a generative paradigm that formulates BioNER as conditional text generation, where entity mentions and their corresponding types are generated directly from input text and task instructions (Keloth et al., 2024). This paradigm offers flexible output representations and facilitates adaptation to diverse entity types and annotation schemas.

Despite recent progress, existing generative BioNER methods still face two key limitations, as illustrated in Figure 1.

Limitation 1: Underutilization of Code-Structural Priors in LLMs. Modern LLMs are pretrained on large-scale corpora that include programming code, enabling them to acquire structural knowledge and code-oriented generation capabilities (Yang et al., 2025a). Previous studies have demonstrated the potential of code-based representations for reasoning and structured information extraction (Gao et al., 2023; Li et al., 2023, 2024b). However, existing generative BioNER methods primarily rely on natural-language instructions or fixed structured output templates, with limited exploration of how code-structural priors can be exploited for entity extraction. Consequently, models may need to learn task-specific output conventions alongside biomedical entity knowledge during supervised fine-tuning (SFT). Given the structured nature of BioNER outputs, code-oriented representations, such as function interfaces, function calls, and structured return objects, offer a promising way to leverage the structural generation capabilities of LLMs. Furthermore, different programming language styles provide diverse structural representations of the same extraction task, potentially reducing dependence on a single output format.

Limitation 2: Misalignment between Sequence-Level Supervision and Entity-Level Objectives. BioNER inherently aims to predict an unordered set of entities, whereas generative methods typically rely on SFT with token-level cross-entropy loss over a linearized reference sequence (Keloth et al., 2024). This

© The Author(s) YEAR. Copyright and licence statements to be updated by the publisher during production.

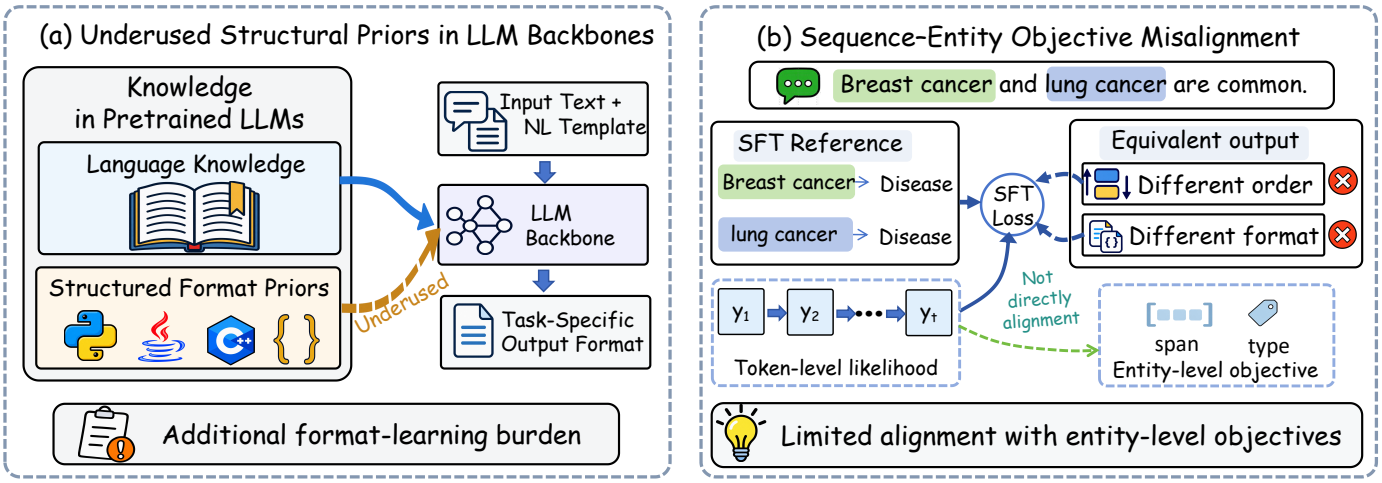


Figure 1 Illustration of limitations of existing generative BioNER methods: (a) underused structural priors in pretrained LLM backbones; (b) Sequence-Entity Objective Misalignment;

discrepancy introduces two related issues. First, the same entity set can be expressed through multiple valid output orders (Madaan et al., 2022), yet conventional SFT usually provides only a single reference sequence. As a result, alternative but semantically equivalent entity orderings receive no explicit positive supervision, potentially introducing a bias toward the reference ordering (Ravikumar et al., 2026). Second, token-level maximum likelihood optimization does not directly reflect entity-level correctness. BioNER is typically evaluated using exact entity matching, where a prediction is correct only if both its boundaries and type match the gold annotation. Therefore, optimizing sequence likelihood alone may not adequately align model training with the entity-level evaluation objective.

To address these limitations, we propose **M**ulti-**C**ode-**S**tyle Supervision and **E**ntity-**L**evel **A**lignment (**MCS-ELA**), a framework for generative BioNER.

To address the **Limitation 1**, MCS-ELA employs *code-structured instruction tuning*, which integrates natural-language task descriptions with function interfaces, function calls, and structured return objects. Specifically, we construct instruction templates in multiple programming language styles, including Python, Java, and C++, to represent the same entity extraction task in different structured forms. This design aims to leverage the code-structural priors of LLMs while exposing the model to diverse output representations.

To address the **Limitation 2**, MCS-ELA improves both supervision diversity and training objective alignment. First, we introduce *multi-order reference augmentation* during SFT, constructing different valid permutations of the same gold entity set to reduce reliance on a fixed reference ordering. Second, we introduce *entity-guided reinforcement learning* based on Group Relative Policy Optimization (GRPO) (Shao et al., 2024). By incorporating reward signals that explicitly account for entity boundary and type correctness, this strategy encourages the model to optimize entity-level extraction quality rather than merely imitate a reference sequence. During inference, predictions generated through different code-style paths are further aggregated via voting to improve robustness.

Our main contributions are summarized as follows:

- We propose MCS-ELA, a generative BioNER framework that incorporates code-structured instruction tuning and multi-code-style supervision to leverage the structural generation capabilities of LLMs.
- We combine multi-order reference augmentation with entity-guided reinforcement learning to mitigate the misalignment between sequence-level supervision and entity-level extraction objectives.
- We conduct extensive experiments on multiple BioNER benchmarks to evaluate the effectiveness of MCS-ELA and its individual components.

2. Related Work

2.1. Biomedical Named Entity Recognition

Biomedical Named Entity Recognition (BioNER) aims to identify biomedical entities from unstructured text. Existing approaches can be broadly categorized into discriminative and generative methods. Discriminative approaches typically formulate BioNER as a sequence labeling task, leveraging domain-specific pretrained language models such as BioBERT (Lee et al., 2020), PubMedBERT (Gu et al., 2021), and BioLinkBERT (Yasunaga et al., 2022). Recent studies have also extended BioNER toward open-domain entity recognition (Yazdani et al., 2026). In contrast, generative approaches formulate entity recognition as conditional text generation, enabling more flexible entity representations and task instructions. Representative studies have explored domain knowledge augmentation (Munnangi et al., 2024), prompting and in-context reasoning (Li et al., 2024a), and instruction tuning (Keloth et al., 2024) to improve extraction performance. While these methods primarily focus on incorporating domain knowledge and improving task adaptation, the potential of leveraging code-structural priors and entity-level optimization for generative BioNER remains less explored.

2.2. LLM-based Information Extraction

Recent advances in large language models (LLMs) have facilitated unified frameworks for information extraction (IE). UIE (Lu et al.,

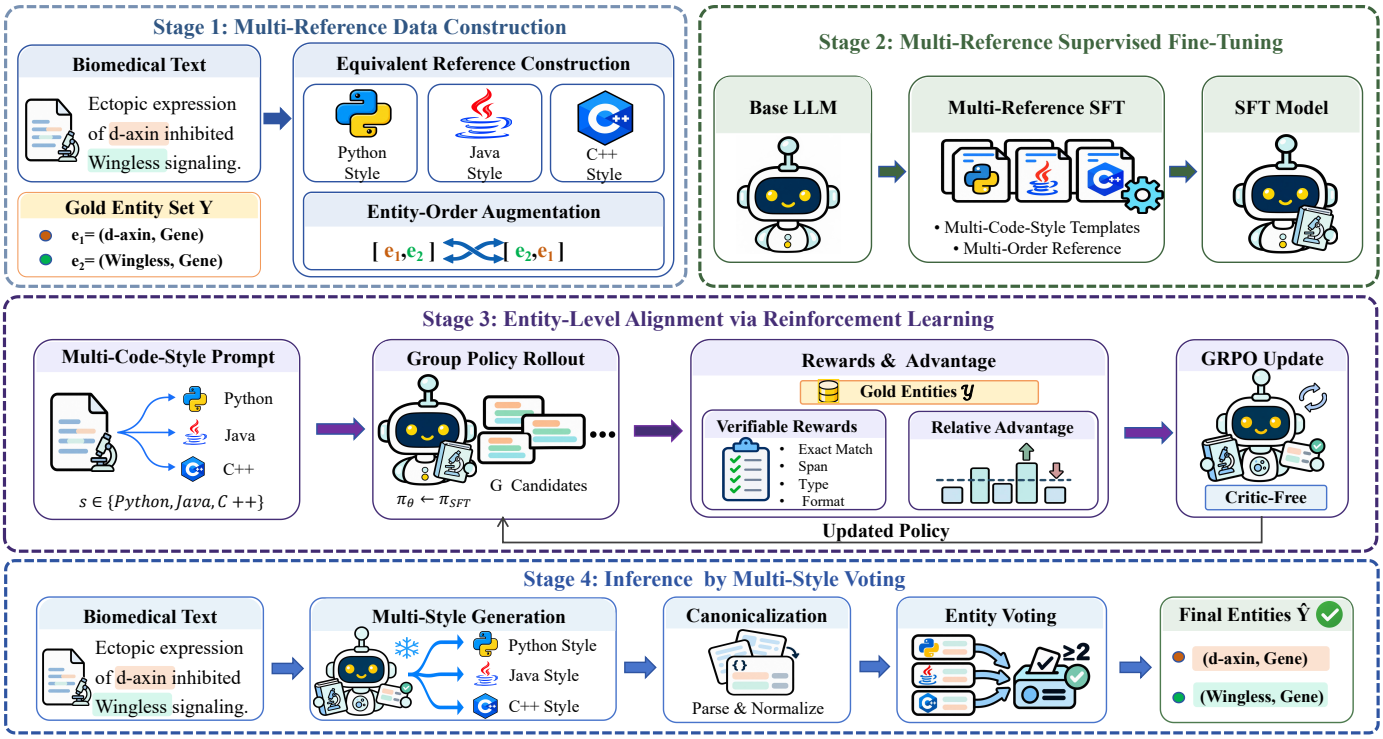


Figure 2 Overview of the MCS-ELA framework.

2022) and InstructUIE (Wang et al., 2023) unify multiple extraction tasks through structured representations and task instructions. Open NER methods further extend entity recognition to diverse or unseen entity types through knowledge distillation and unified entity taxonomies (Zhou et al., 2024; Yang et al., 2025b). To improve structured extraction, CodeIE (Li et al., 2023), KnowCoder (Li et al., 2024b), and MPL (Li et al., 2025) introduce code-style representations to specify extraction schemas and outputs, while GoLLIE (Sainz et al., 2024) and GuideNER (Huang et al., 2025) incorporate annotation guidelines to enhance task understanding and generalization. More recently, reinforcement learning with GRPO and structured-unit rewards has been investigated for information extraction (Liu et al., 2026). Building on these advances, our work explores multi-code-style supervision and entity-guided reinforcement learning for generative BioNER, with a particular focus on mitigating the mismatch between sequence-level supervision and entity-level extraction objectives.

3. Method

3.1. Task Formulation

Given a biomedical text T and a predefined entity type set C, BioNER aims to extract an entity set $Y = \{(s_i, t_i)\}_{i=1}^{N}$. Each s_i is an entity text span in T, $t_i \in C$ is its type. We formulate BioNER as conditional generation. Given T and a task instruction I that specifies the target entity types, the model generates a structured representation of the predicted entity set $\hat{Y}$.

Table 1 Statistics of the BioNER corpora.

Dataset	Entity type	Train	Dev	Test
NCBI-Disease	Disease	5,424	923	940
BC5CDR	Disease, Chemical	4,560	4,581	4,797
BC4CHEMD	Chemical	30,682	30,639	26,364
BC2GM	Gene	12,500	2,500	5,000
JNLPBA	Five types	18,607	1,939	4,260

BC5CDR is evaluated separately for disease and chemical recognition using the same text splits. JNLPBA includes five entity types: Protein, DNA, RNA, Cell Line, and Cell Type.

3.2. Overview

We propose **M**ulti-**C**ode-**S**tyle Supervision and **E**ntity-**L**evel **A**lignment (**MCS-ELA**), a framework for generative BioNER. As shown in Figure 2, MCS-ELA learns to generate biomedical entity sets through equivalent-reference supervision and entity-level feedback. For each annotated text, we construct Python, Java, and C++ style references and vary the entity order while preserving the annotated spans and types. These references are used for supervised fine-tuning. We then refine the model with GRPO using rewards computed from predicted and gold entity sets. At inference time, predictions from the three code styles are combined by voting.

3.3. Multi-Reference Data Construction

Multi-code-style representation. We use three code styles, $\mathcal{L} = \{\text{Python}, \text{Java}, \text{C++}\}$. Each template contains a natural language task description, the entity type set, an entity object definition, and task function interfaces. Each entity object has two fields, `span` and `type`. The interfaces specify entity extraction and type classification,

Table 2 Micro-F1 scores (%) on six BioNER settings.

Category	Method	Backbone	NCBI (Disease)	BC5CDR (Disease)	BC5CDR (Chemical)	BC4CHEMD (Chemical)	BC2GM (Gene)	JNLPBA (Multiple types)
Biomedical PLMs	BioBERT#	BioBERT	88.12	83.83	91.98	89.55	81.94	76.23
	PubMedBERT#	PubMedBERT	86.92	86.07	93.60	89.91	83.36	76.19
	BioLinkBERT*	BioLinkBERT-large	88.76	86.39	**94.04**	89.92	85.18	76.35
	OpenBioNER#	BioBERT-base	86.10	–	–	88.50	80.50	74.30
LLM-based labeling	VANER*	LLaMA-2-7B	88.03	86.56	92.84	91.32	83.25	75.92
General-purpose LLMs	Zero-shot inference	GPT 5.6 Sol	73.05	64.29	83.58	62.16	67.15	62.34
		DeepSeek V4.1 Flash	69.56	59.47	80.63	56.21	61.07	56.92
		Qwen 3.7 Flash	66.87	60.63	79.60	54.23	54.71	52.95
Generative BioNER	BioNER-LLaMA1*	LLaMA-1-7B	87.60	86.42	92.60	90.23	83.90	75.78
	NERLlama3.1*	Llama-3.1-8B-Instruct	88.25	86.64	93.09	91.02	84.26	76.04
	MCS-ELA (Ours)	Qwen3-4B	88.42	87.05	93.64	92.19	85.31	76.42
	MCS-ELA (Ours)	Qwen3-8B	**88.96**	**87.54**	93.97	**92.69**	**85.93**	**76.98**

Bold indicates the highest F1 in each column. $*$ denotes results obtained by retraining and evaluating the baselines in this work; # denotes results reported in the original publications. Zero-shot LLM results are obtained by direct prompting without task-specific fine-tuning or in-context demonstrations. The VANER reproduction excludes Dense Bioentities Recognition and its knowledge-base-derived auxiliary training examples.

while a function call provides the input text. The reference output is a sequence of entity objects written in the corresponding programming language.

All three styles use the same input text and entity types, and encode the same gold entities. For each style $\ell \in \mathcal{L}$, a converter $\mathcal{F}_\ell$ maps the text T, task instruction I, and gold entity set Y to a prompt X_ℓ and a reference output R_ℓ:

$$(X_\ell, R_\ell) = \mathcal{F}_\ell(T, I, Y). \tag{1}$$

This construction preserves the task meaning across styles while exposing the model to different code structures.

Entity order augmentation. we randomly shuffle the gold entities for each instance at every training epoch, where τ denotes the sampled order:

$$\mathbf{y}_\tau = \mathrm{RandomShuffle}(Y). \tag{2}$$

We convert $\mathbf{y}_\tau$ into a reference output $R_{\ell,\tau}$ in code style ℓ. The input prompt X_ℓ remains unchanged. Although the references differ in syntax and entity order, their corresponding parsers recover the same gold entity set Y. Code-style conversion and entity order augmentation therefore provide multiple semantically equivalent references for each instance.

3.4. Multi-Reference Supervised Fine-Tuning

We fine-tune Qwen3-4B and Qwen3-8B separately (Yang et al., 2025a). Each model is trained using references from all three code styles and different entity orders. Let $\mathcal{D}_{\mathrm{aug}}$ contain the resulting training pairs $(X_\ell, R_{\ell,\tau})$. For $(X, R) \sim \mathcal{D}_{\mathrm{aug}}$, write the reference of M tokens as $R = (r_1, \ldots, r_M)$, with $r_{<m}$ denoting its prefix before position m. We optimize model parameters θ using autoregressive cross-entropy:

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\mathbb{E}\left[\frac{1}{M}\sum_{m=1}^{M} \log p_\theta(r_m \mid X, r_{<m})\right]. \tag{3}$$

All references are valid training targets for the same entity set. This training aims to reduce dependence on a fixed output format or entity order. For each backbone, the resulting SFT model initializes the RL policy.

3.5. Entity-Level Alignment via Reinforcement Learning

SFT learns entity extraction through reference token prediction, but does not explicitly compare candidate outputs by entity-level correctness. We therefore apply entity-guided RL with GRPO using rewards based on entity-set matching.

Candidate generation. The SFT model initializes π_θ and remains frozen as π_{ref}. Each rollout sets $\pi_{\mathrm{old}} \leftarrow \pi_\theta$. For each prompt $X_{j,\ell}$, π_{old} samples G candidate outputs o_j^i. This process is repeated for all three code styles. Each output is parsed into an entity set $\hat{Y}_j^i$ for reward computation against the gold set Y_j.

Verifiable rewards. We combine entity and format rewards:

$$r_j^i = r_{j,\mathrm{ent}}^i + \lambda_{\mathrm{fmt}} r_{j,\mathrm{fmt}}^i, \tag{4}$$

where λ_{fmt} controls the contribution of the format reward.

The entity reward combines strict entity matching with partial boundary matching. Strict matching requires both the entity span and type to match the gold annotation, regardless of entity order. It yields entity-level F1 F and the counts FP and FN for spurious and missed entities.

To provide partial credit for boundary overlap, we greedily match predicted and gold entities of the same type using token-overlap F1, denoted by F_{tok}. The resulting matched pairs form $\mathcal{M}_j^i$, and the soft boundary score is

$$B = \frac{2\sum_{(e,g)\in\mathcal{M}_j^i} F_{\mathrm{tok}}(e,g)}{|\hat{Y}_j^i| + |Y_j|}. \tag{5}$$

We combine the two matching scores with penalties for missed and spurious entities:

$$q_j^i = \alpha_F F + \alpha_B B - \lambda_{\mathrm{FN}} \frac{\mathrm{FN}}{\max(|Y_j|, 1)} - \lambda_{\mathrm{FP}} \frac{\mathrm{FP}}{\max(|\hat{Y}_j^i|, 1)}. \tag{6}$$

The entity reward is generally obtained by clipping q_j^i to $[-1, 1]$, with separate rules for invalid, empty, or unmatched outputs.

The format reward measures whether the output can be parsed into entity objects. It is 1 for directly parseable outputs, 0.5 when an entity list can be recovered from the output, and 0 otherwise. This reward is computed separately from the entity reward, and the total reward is not clipped.

Group-relative policy updates. We use the combined entity and format rewards to guide policy optimization. For candidates generated from the same prompt, we normalize their rewards within the group to obtain advantages $\hat{A}_j^i = (r_j^i - \mu_j)/(\sigma_j + \delta)$. This encourages candidates with above-average rewards and discourages those with below-average rewards.

During optimization, we compare the current policy with the old policy that generated the candidates through the token-level probability ratio:

$$\rho_{j,t}^i = \frac{\pi_\theta(o_{j,t}^i \mid X_{j,\ell}, o_{j,<t}^i)}{\pi_{\text{old}}(o_{j,t}^i \mid X_{j,\ell}, o_{j,<t}^i)}. \tag{7}$$

To stabilize updates, we clip this ratio to $[1-\epsilon, 1+\epsilon]$, obtaining $\bar{\rho}_{j,t}^i$, and penalize deviation from the frozen SFT reference policy through KL regularization. The resulting loss is

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}\Big[\min\Big(\rho_{j,t}^i \hat{A}_j^i, \bar{\rho}_{j,t}^i \hat{A}_j^i\Big)\Big] + \beta \widehat{D}_{\text{KL}}. \tag{8}$$

This objective increases the likelihood of candidates with higher relative entity and format rewards while controlling policy drift. It provides direct task-level feedback on biomedical mention boundaries, entity types, and missed entities.

3.6. Multi-Style Voting Inference

We use multi-style voting to combine predictions from the three code styles. For each input text, the RL-trained policy π_{RL} generates one structured output using each of the Python, Java, and C++ prompts. We parse the outputs, normalize entity spans and type labels, and remove duplicate span–type pairs within each style to obtain entity sets $\hat{Y}_\ell$, where $\ell \in \mathcal{L}$. The final prediction retains entity span–type pairs supported by at least two of the three styles. Each style contributes at most one vote per pair, and agreement requires both the normalized span and type to match.

4. Experiments and Analysis

4.1. Datasets and Evaluation Metrics

We evaluate MCS-ELA on five biomedical corpora covering six BioNER settings. NCBI-Disease (Doğan et al., 2014) targets disease entities. BC5CDR (Li et al., 2016) provides both disease and chemical entities and is evaluated in two separate recognition settings. BC4CHEMD (Krallinger et al., 2015) targets chemical entities; BC2GM (Smith et al., 2008) targets gene entities; and JNLPBA (Collier et al., 2004) contains five entity types. Table 1 reports the corpus statistics and the train, development, and test partitions used in our experiments after preprocessing.

We adopt the strict matching criterion: a predicted entity is considered correct only when both its boundaries and type exactly match the gold annotation. We report micro-averaged precision, recall, and F1:

Table 3 Ablation results of MCS-ELA.

Method	BC5CDR(Disease)	BC4CHEMD(Chemical)	BC2GM(Gene)
MCS-ELA	87.54	92.69	85.93
w/o Multi-Code	86.50	91.32	83.87
w/o Code	86.43	91.26	83.76
w/o Multi-Order	86.59	92.05	84.33
w/o RL	86.48	91.83	84.27
w/o Format Reward	87.03	91.97	84.34

w/o Multi-Order uses a fixed entity order. w/o RL uses only SFT, while w/o Format Reward removes the format reward. w/o Multi-Code uses a single Python template. w/o Code uses a natural-language template. Both retain entity-order augmentation and the full RL reward.

Table 4 Results of different code styles and aggregation methods.

Inference	BC5CDR(Disease)	BC4CHEMD(Chemical)	BC2GM(Gene)
Python	86.53	92.01	84.07
Java	86.98	92.18	84.55
C++	86.67	91.86	84.21
Union	87.34	92.64	85.67
Voting	87.54	92.69	85.93

Python, Java, and C++ denote single-style inference. Union merges the three predicted entity sets and removes duplicates. Voting selects entities using the rule described in the method section.

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}. \tag{9}$$

where TP, FP, and FN denote the numbers of correctly predicted, incorrectly predicted, and missed entities, respectively. We use F1 as the primary evaluation metric.

4.2. Experimental Settings and Baselines

We implement MCS-ELA using PyTorch and MS-SWIFT, with Qwen3-4B[1] and Qwen3-8B[2] as backbone models (Yang et al., 2025a). We use LoRA (Hu et al., 2022) with a rank of 8 and a scaling factor α of 32. For SFT, we train the models for 5 epochs using AdamW with a learning rate of 2×10^{-4}, a per-device batch size of 4, and 2 gradient accumulation steps. We then perform GRPO training for 2 epochs with a learning rate of 5×10^{-6} and sample 4 responses for each input. All experiments are conducted on a single NVIDIA RTX PRO 6000 GPU. We select the best checkpoint based on development-set F1 and report its corresponding test performance.

We compare MCS-ELA with representative BioNER methods using published results and our evaluations. The baselines fall into four groups. (1) **Biomedical PLM-based methods** include sequence-labeling models BioBERT (Lee et al., 2020), PubMedBERT (Gu et al., 2021), and BioLinkBERT (Yasunaga et al., 2022), as well as the description-conditioned cross-encoder OpenBioNER (Cocchieri et al., 2025). (2) **LLM-based sequence-labeling methods** use an LLM backbone to predict token labels. We reproduce VANER without Dense Bioentities Recognition (DBR) (Bian et al., 2024), excluding its knowledge-base-derived auxiliary training examples. (3) **General-purpose**

[1] https://huggingface.co/Qwen/Qwen3-4B

[2] https://huggingface.co/Qwen/Qwen3-8B

LLMs include GPT-5.6 Sol[3], DeepSeek-V4.1-Flash[4], and Qwen3.7-Flash[5], evaluated under zero-shot inference. (4) **Generative BioNER methods** produce entity mentions and types as model outputs. We include BioNER-LLaMA1 (Keloth et al., 2024) and NERLlama3.1 (Liu et al., 2025). Our task-trained baseline reproductions use the data splits and strict entity-matching criterion described above.

4.3. Main Results

To evaluate the effectiveness of MCS-ELA, we compare it with BioNER baselines from four categories: biomedical pretrained models, an LLM-based sequence-labeling method, generative BioNER methods, and general-purpose LLMs under zero-shot inference. As shown in Table 2, MCS-ELA achieves competitive performance across the six settings.

(1) Compared with biomedical pretrained models, MCS-ELA achieves competitive results on all six BioNER settings. MCS-ELA specifies entity types through task instructions and uses a unified structured generation approach. This can reduce the need for dataset-specific models.

(2) LLM-based sequence-labeling methods such as VANER predict a label for each input token. MCS-ELA directly generates entity mentions and their types. Its strong results suggest that structured generation can support accurate boundary and type prediction while allowing flexible output formats.

(3)With a 4B backbone, MCS-ELA outperforms the evaluated open- and closed-source LLMs under zero-shot inference. This suggests that task-specific training and entity-level alignment can enable strong BioNER performance with smaller models.

(4) MCS-ELA improves entity extraction performance over the listed generative BioNER baselines. Multi-code-style supervision and entity-order augmentation are designed to exploit the structural priors of LLMs and reduce dependence on fixed reference formats and orders. Entity-guided RL and multi-style voting complement this supervision through entity-level feedback during training and cross-style agreement during inference.

(5) The 8B variant of MCS-ELA outperforms its 4B counterpart on all six settings. Larger models may better understand complex biomedical text and follow code-style instructions. This may help them extract complete multi-word entity mentions and predict their types accurately while following the required output format.

4.4. Ablation Study

To evaluate the contribution of each component, we conduct ablation studies on BC5CDR-Disease, BC4CHEMD, and BC2GM. The comparisons examine code-style representation, multi-code-style supervision, entity-order augmentation, RL, and the format reward. Table 3 reports the results of the full model and its ablated variants.

(1) The single-Python and natural-language variants test the role of template representation, while single- and multi-code training test supervision diversity. The drop of w/o Multi-Code suggests that different code formats provide useful supervision for the same entity annotations.

[3] https://developers.openai.com/api/docs/models/gpt-5.6-sol

[4] https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash

[5] https://www.alibabacloud.com/help/en/model-studio/qwen3-7-flash

Table 5 Separate and mixed training results.

Setting	Separate Training	Mixed Training
NCBI-Disease	83.23	85.83
BC5CDR-Disease	79.43	82.03
BC5CDR-Chemical	87.42	89.34
BC4CHEMD-Chemical	78.84	80.1
BC2GM-Gene	77.87	82.46

Both strategies use the same sampled instances for each setting. Evaluation follows the entity types and annotations of each dataset.

(2) Entity-order augmentation also improves extraction performance. The drop of w/o Multi-Order suggests that varying valid entity orders helps reduce dependence on a fixed reference order. This is consistent with BioNER evaluation, which depends on entity boundaries and types rather than output order.

(3) RL further improves extraction performance after SFT. The full model outperforms the SFT-only variant. While SFT learns reference sequences, RL provides feedback on the correctness of predicted entities. This helps align training with the strict matching criterion in BioNER.

(4) The drop of w/o Format Reward suggests that explicit format constraints remain useful during RL. This reward helps the model follow the required code format and reduces the risk that format errors prevent the parser from reading predicted entities and their types.

4.5. Multi-Code-Style Representations and Voting

To study the effects of code styles and voting, we use the same MCS-ELA model to generate predictions with Python, Java, and C++ prompts. Table 4 reports the results on the three BioNER settings. The three code styles show differences in performance. Different styles may miss different entities or produce different false positives. Voting outperforms single-style inference and Union. This suggests that voting can improve extraction by selecting from the same candidate predictions. Agreement on entity boundaries and types across code styles may reduce false positives introduced by a single style.

4.6. Experiments under Limited Supervision

4.6.1. Low-Resource Analysis

Biomedical entity annotation is costly. To evaluate how effectively MCS-ELA uses limited annotations, we conduct low-resource experiments with Qwen3-8B on BC5CDR-Disease, BC4CHEMD, and BC2GM. We use 25%, 50%, 75%, and 100% of the training instances and evaluate on the full test set of each dataset. All compared configurations share the same sampled instances. As shown in Figure 3, the results suggest that code-based representations, equivalent references, and RL help the model make better use of limited annotations.

4.6.2. Multi-Dataset Training

To examine whether supervision from other datasets improves BioNER under limited annotations, we conduct multi-dataset training experiments with Qwen3-8B. We sample 25% of the training instances from each of five BioNER settings and compare separate training with mixed training. As shown in Table 5, mixed training improves performance on all five settings. These results support using

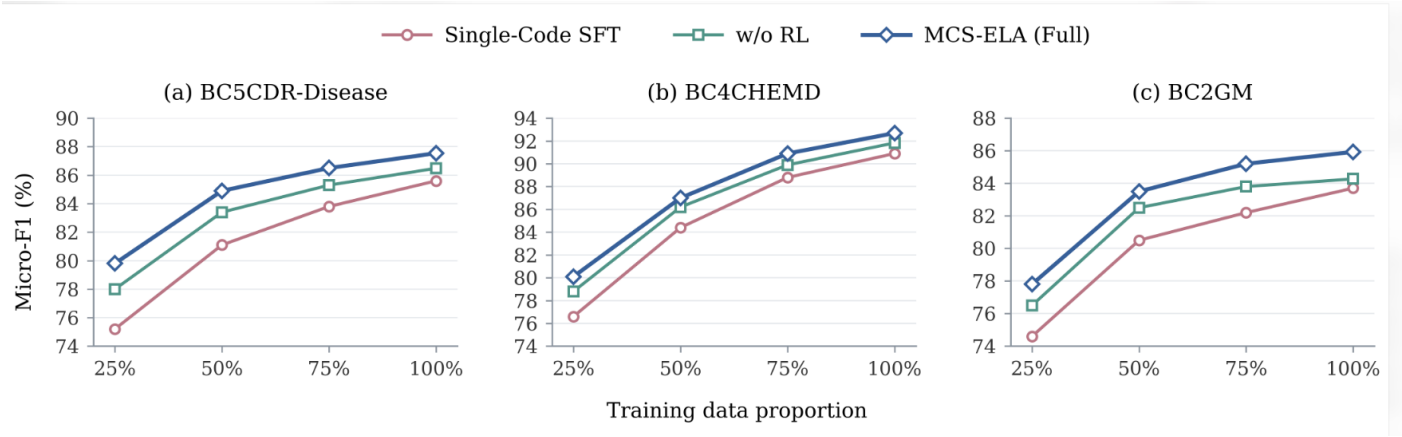


Figure 3 Low-resource BioNER results. We compare Single-Code SFT, MCS-ELA without reinforcement learning (w/o RL), and the full model.

cross-dataset supervision when annotations for the target setting are limited. Terms and contexts from different sources may help the shared model learn more transferable entity extraction capabilities.

5. Conclusion

We propose MCS-ELA, a generative BioNER framework that combines multi-code-style supervision with entity-level alignment through entity-guided reinforcement learning. The framework formulates BioNER as code-style structured generation to make better use of the structural priors of LLMs. Multi-code-style supervision and entity-order augmentation reduce dependence on a fixed reference format and order. After supervised fine-tuning, we use GRPO to further optimize the model with verifiable rewards for entity and format. During inference, multi-style voting combines predictions to improve their reliability. Experiments on six BioNER settings show competitive performance, and ablation studies validate the effectiveness of each component.

6. Acknowledgments

We thank the anonymous reviewers for their valuable suggestions. This work was supported by the National Natural Science Foundation of China (Grant Nos. 62572089 and 62506058) and the Young Teachers Science and Technology Innovation Project under the Fundamental Research Funds for the Central Universities (Grant No. 3132026362).

References

J. Bian, W. Zhai, X. Huang, J. Zheng, and S. Zhu. VANER: Leveraging large language model for versatile and adaptive biomedical named entity recognition. In *European Conference on Artificial Intelligence*, volume 392, pages 1583–1590, 2024.

A. Cocchieri, G. Frisoni, M. Martínez Galindo, G. Moro, G. Tagliavini, and F. Candoli. OpenBioNER: Lightweight open-domain biomedical named entity recognition through entity type description. In *Findings of the Association for Computational Linguistics*, pages 818–837, 2025.

N. Collier, T. Ohta, Y. Tsuruoka, Y. Tateisi, and J.-D. Kim. Introduction to the bio-entity recognition task at JNLPBA. In *International Joint Workshop on Natural Language Processing in Biomedicine and its Applications*, pages 73–78, 2004.

R. I. Doğan, R. Leaman, and Z. Lu. NCBI disease corpus: A resource for disease name recognition and concept normalization. *Journal of Biomedical Informatics*, 47:1–10, 2014.

L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning*, volume 202, pages 10764–10799, 2023.

Y. Gu, R. Tinn, H. Cheng, M. Lucas, N. Usuyama, X. Liu, T. Naumann, J. Gao, and H. Poon. Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare*, 3(1):1–23, 2021.

M. Habibi, L. Weber, M. Neves, D. L. Wiegandt, and U. Leser. Deep learning with word embeddings improves biomedical named entity recognition. *Bioinformatics*, 33(14):i37–i48, 2017.

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

S. Huang, B. Xu, Y. Yu, C. Li, and X. A. Lin. GuideNER: Annotation guidelines are better than examples for in-context named entity recognition. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(23):24159–24166, 2025.

V. K. Keloth, Y. Hu, Q. Xie, X. Peng, Y. Wang, A. Zheng, M. Selek, K. Raja, C.-H. Wei, Q. Jin, Z. Lu, Q. Chen, and H. Xu. Advancing entity recognition in biomedicine via instruction tuning of large language models. *Bioinformatics*, 40(4):btae163, 2024.

M. Krallinger, O. Rabal, F. Leitner, M. Vazquez, D. Salgado, et al. The CHEMDNER corpus of chemicals and drugs and its annotation principles. *Journal of Cheminformatics*, 7(Suppl 1):S2, 2015.

T. Lai, H. Ji, C. Zhai, and Q. H. Tran. Joint biomedical entity and relation extraction with knowledge-enhanced collective inference. In *Annual Meeting of the Association for Computational Linguistics and International*

Joint Conference on Natural Language Processing, pages 6248–6260, 2021.

J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J. Kang. BioBERT: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4): 1234–1240, 2020.

B. Li, G. Fang, W. Ye, Z. Xu, J. Zhang, H. Cheng, and S. Zhang. MPL: Multiple programming languages with large language models for information extraction. In *Findings of the Association for Computational Linguistics*, pages 2403–2414, 2025.

J. Li, Y. Sun, R. J. Johnson, D. Sciaky, C.-H. Wei, R. Leaman, A. P. Davis, C. J. Mattingly, T. C. Wiegers, and Z. Lu. BioCreative V CDR task corpus: a resource for chemical disease relation extraction. *Database*, 2016:baw068, 2016.

M. Li, H. Zhou, H. Yang, and R. Zhang. RT: a Retrieving and Chain-of-Thought framework for few-shot medical named entity recognition. *Journal of the American Medical Informatics Association*, 31(9):1929–1938, 2024a.

P. Li, T. Sun, Q. Tang, H. Yan, Y. Wu, X. Huang, and X. Qiu. CodeIE: Large code generation models are better few-shot information extractors. In *Annual Meeting of the Association for Computational Linguistics*, pages 15339–15353, 2023.

Z. Li, Y. Zeng, Y. Zuo, W. Ren, W. Liu, M. Su, Y. Guo, Y. Liu, X. Li, Z. Hu, L. Bai, W. Li, Y. Liu, P. Yang, X. Jin, J. Guo, and X. Cheng. KnowCoder: Coding structured knowledge into LLMs for universal information extraction. In *Annual Meeting of the Association for Computational Linguistics*, pages 8758–8779, 2024b.

H. Liu, Z. Chen, P. Li, Y.-Z. Liu, X. Liu, R. X. Xu, and M. Sun. Resource-efficient instruction tuning of large language models for biomedical named entity recognition. *Journal of Biomedical Informatics*, 170:104896, 2025.

W. Liu, Z. Song, S. Nie, G. Liu, L. Chen, B. Yang, Y. Chen, P. Zhou, H. Wang, Y. Liu, W. Hu, J. Xu, R. Shi, and Y. Huang. ProUIE: A macro-to-micro progressive learning method for LLM-based universal information extraction. In *Findings of the Association for Computational Linguistics*, pages 21737–21750, 2026.

Y. Lu, Q. Liu, D. Dai, X. Xiao, H. Lin, X. Han, L. Sun, and H. Wu. Unified structure generation for universal information extraction. In *Annual Meeting of the Association for Computational Linguistics*, pages 5755–5772, 2022.

A. Madaan, D. Rajagopal, N. Tandon, Y. Yang, and A. Bosselut. Conditional set generation using seq2seq models. In *Conference on Empirical Methods in Natural Language Processing*, pages 4874–4896, 2022.

M. Munnangi, S. Feldman, B. Wallace, S. Amir, T. Hope, and A. Naik. On-the-fly definition augmentation of LLMs for biomedical NER. In *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3833–3854, 2024.

R. Ravikumar, N. Ibrahim, and R. Batista-Navarro. Lost in formatting: How output formats skew LLM performance on information extraction. In *Conference of the European Chapter of the Association for Computational Linguistics*, pages 5498–5513, 2026.

O. Sainz, I. García-Ferrero, R. Agerri, O. Lopez de Lacalle, G. Rigau, and E. Agirre. GoLLIE: Annotation guidelines improve zero-shot information-extraction. In *International Conference on Learning Representations*, 2024.

Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

L. Smith, L. K. Tanabe, R. J. Ando, C.-J. Kuo, I.-F. Chung, C.-N. Hsu, Y.-S. Lin, R. Klinger, C. M. Friedrich, K. Ganchev, M. Torii, H. Liu, B. Haddow, C. A. Struble, R. J. Povinelli, A. Vlachos, W. A. Baumgartner, Jr., L. Hunter, B. Carpenter, R. T.-H. Tsai, H.-J. Dai, F. Liu, Y. Chen, C. Sun, S. Katrenko, P. Adriaans, C. Blaschke, R. Torres, M. Neves, P. Nakov, A. Divoli, M. Maña-López, J. Mata, and W. J. Wilbur. Overview of BioCreative II gene mention recognition. *Genome Biology*, 9(Suppl 2):S2, 2008.

B. Song, F. Li, Y. Liu, and X. Zeng. Deep learning methods for biomedical named entity recognition: a survey and qualitative comparison. *Briefings in Bioinformatics*, 22(6):bbab282, 2021.

X. Wang, W. Zhou, C. Zu, H. Xia, T. Chen, Y. Zhang, R. Zheng, J. Ye, Q. Zhang, T. Gui, J. Kang, J. Yang, S. Li, and C. Du. InstructUIE: Multi-task instruction tuning for unified information extraction. *arXiv preprint arXiv:2304.08085*, 2023.

R. Xu, W. Shi, Y. Yu, Y. Zhuang, Y. Zhu, M. D. Wang, J. C. Ho, C. Zhang, and C. Yang. BMRetriever: Tuning large language models as better biomedical text retrievers. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 22234–22254, 2024.

A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.

Y. Yang, W. Zhao, C. Huang, J. Ye, X. Wang, H. Zheng, Y. Nan, Y. Wang, X. Xu, K. Huang, Y. Zhang, T. Gui, Q. Zhang, and X. Huang. Beyond boundaries: Learning a universal entity taxonomy across datasets and languages for open named entity recognition. In *International Conference on Computational Linguistics*, pages 10902–10923, 2025b.

M. Yasunaga, J. Leskovec, and P. Liang. LinkBERT: Pretraining language models with document links. In *Annual Meeting of the Association for Computational Linguistics*, pages 8003–8016, 2022.

A. Yazdani, I. Stepanov, and D. Teodoro. GLiNER-BioMed: a suite of efficient models for open biomedical named entity recognition. *Bioinformatics*, 42(6):btag322, 2026.

W. Zhou, S. Zhang, Y. Gu, M. Chen, and H. Poon. UniversalNER: Targeted distillation from large language models for open named entity recognition. In *International Conference on Learning Representations*, 2024.